

Memoria del proyecto

Clasificación de solicitudes de préstamo

Proyecto de Aprendizaje Automático · Curso 2025-2026

Autor: Pablo Aguilar García

GitHub: [loan_approval](#)

Idea clave

Se establece una línea base con Regresión logística y se analiza la mejora obtenida al aumentar la complejidad mediante redes neuronales y redes profundas. Además, se estudia el desbalanceo de clases comparando entrenamiento sin balanceo, undersampling y oversampling con SMOTE. Los resultados muestran que el problema tiene una estructura simple y casi lineal: la regresión logística ya alcanza un rendimiento muy alto, mientras que las redes neuronales apenas aportan mejoras pese a su mayor complejidad. También se observa que el tamaño del dataset es suficiente, ya que SMOTE no produce mejoras relevantes.

Índice

1. Introducción y objetivos
2. Descripción y análisis del conjunto de datos
3. Preprocesado y limpieza de datos
4. Técnicas y modelos aplicados
5. Metodología experimental y métricas
6. Curvas de diagnóstico
7. Resultados experimentales
8. Comparativa y análisis crítico
9. Limitaciones y mejoras futuras
10. Reproducibilidad

1. Introducción y objetivos

En este proyecto se aborda un problema de clasificación binaria orientado a predecir la aprobación de un préstamo a partir de ciertas características del solicitante. Para la implementación se usa un conjunto de datos público de la web de Kaggle llamado [Loan Approval Classification Dataset](#).

A partir de variables personales, financieras y crediticias, el sistema debe predecir la variable objetivo `loan_status`. El interés del problema está en comparar varias técnicas de aprendizaje automático y analizar si modelos más complejos aportan una mejora real frente a modelos más simples.

El trabajo se ha planteado con un enfoque experimental. Primero se construye una línea base con regresión logística, después se compara con una red neuronal implementada manualmente con NumPy, y finalmente se añade una red neuronal profunda implementada con PyTorch como técnica adicional. También se estudia el desbalanceo con dos estrategias: submuestreo de la clase mayoritaria y oversampling con SMOTE.

Objetivos

11. Aplicar un flujo completo de aprendizaje automático: análisis exploratorio, preprocesado, entrenamiento, validación y evaluación final.
12. Comparar varias técnicas estudiadas en la asignatura, incluyendo una red neuronal implementada con NumPy.
13. Evaluar los modelos con métricas adecuadas para clasificación y con validación cruzada estratificada.
14. Analizar el efecto del desbalanceo de clases usando entrenamiento original, undersampling y oversampling con SMOTE.
15. Construir curvas de entrenamiento, learning curves y validation curves para diagnosticar sesgo, varianza y sensibilidad a hiperparámetros.

2. Descripción y análisis del conjunto de datos

El dataset original contiene 45.000 ejemplos y 14 columnas. La variable objetivo es `loan_status`. El resto de variables describen información del solicitante y del préstamo: edad, género, nivel educativo, ingresos, experiencia laboral, tipo de vivienda, cantidad solicitada, finalidad del préstamo, tipo de interés, porcentaje de ingresos destinado al préstamo, historial crediticio, puntuación crediticia y existencia de defaults previos.

Edad	Numérica	-
Género	Categórica	{Male, Female}
Educación	Categórica	{Bachelor, Associate, High School, Master, Doctorate}
Salario	Numérica	-
Experiencia Laboral	numérica	-
Vivienda (ownership)	Categórica	{Rent, Mortgage, Own, Other}

Cantidad solicitada	Numérica	-
Propósito	Categórica	{Education, Medical, Venture, Personal, Debt Consolidation, Home Improvement}
Interés	Numérica	-
Ratio	Numérica	-
Ant Hist Crediticio	Numérica	-
Credit Score	Numérica	-
Prestamos Anteriores	Categórica	{No, Yes}
Estado (Etiqueta)	Binaria	{0, 1}

Tabla 1. Tipo de cada variable y sus posibles valores.

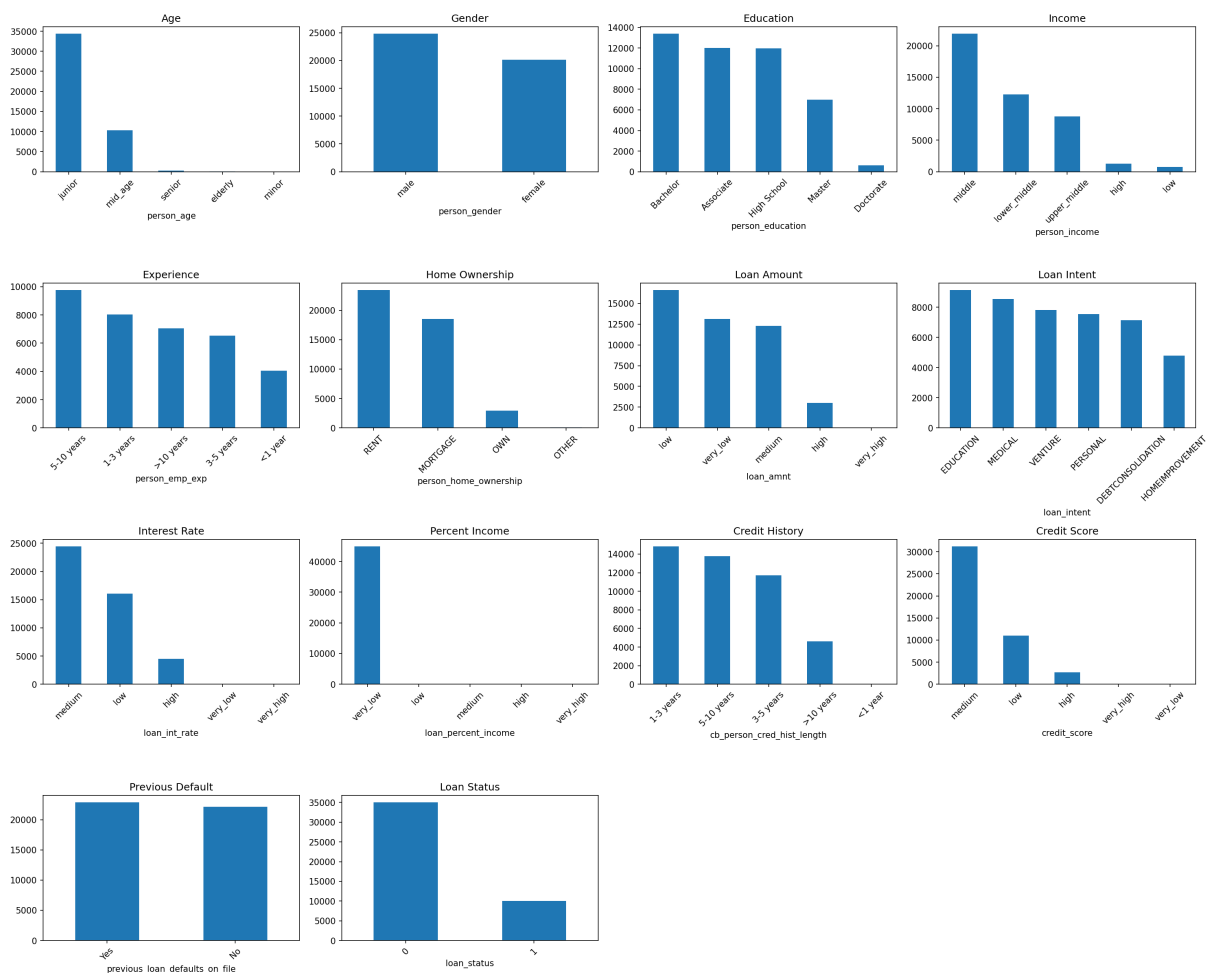


Figura 1. Distribuciones principales de las variables del dataset.

El análisis exploratorio muestra que varias variables presentan distribuciones concentradas. La mayoría de solicitantes se agrupan en edades jóvenes, los ingresos se concentran en rangos medios y el porcentaje de ingresos destinado al préstamo aparece mayoritariamente en valores bajos.

Desbalanceo de la variable objetivo

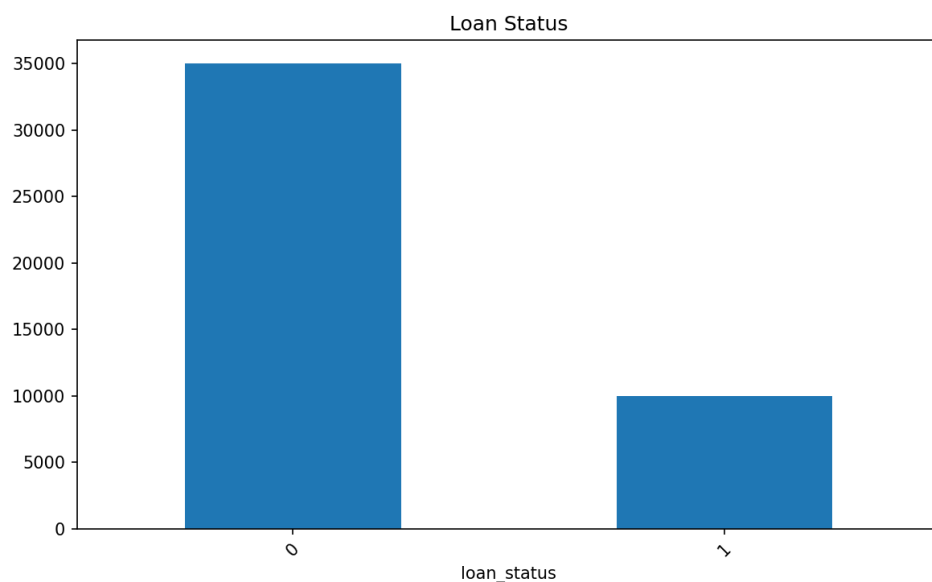


Figura 2. Distribución de la variable objetivo `loan_status`.

La variable objetivo no está equilibrada: la clase 0 es mayoritaria frente a la clase 1. Este desbalanceo justifica que no se use únicamente accuracy como métrica, ya que un modelo podría obtener una exactitud elevada favoreciendo la clase mayoritaria.

Variable de defaults previos

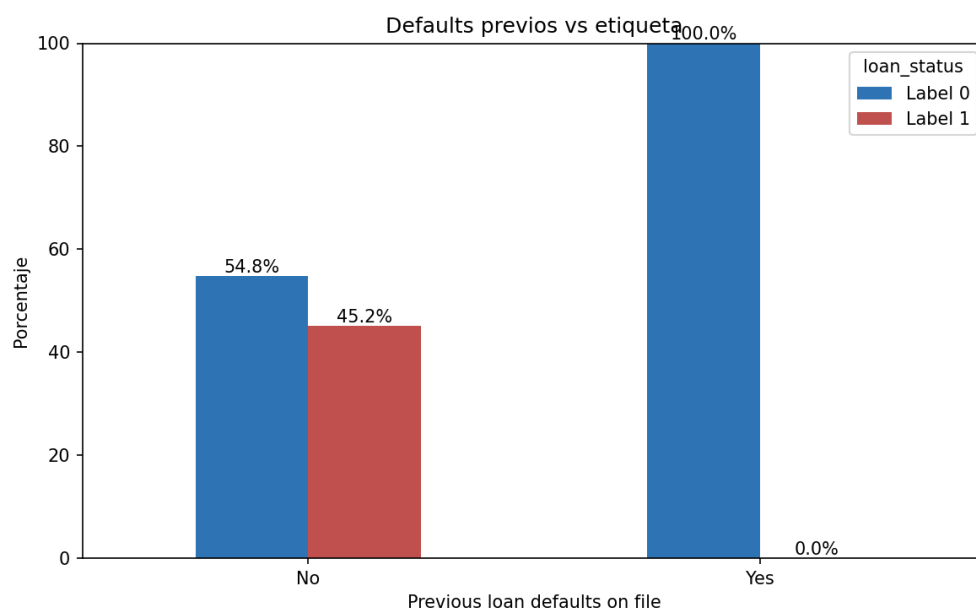


Figura 3. `Previous_defaults_on_file` vs `label`.

La variable `previous_loan_defaults_on_file` tiene gran relevancia predictiva. Durante el desarrollo se exploró si podía utilizarse como regla determinista, pero finalmente se descartó ese enfoque porque el modelo ya era capaz de aprovechar la información de esta variable dentro del entrenamiento normal.

3. Preprocesado y limpieza de datos

El preprocesado se ha diseñado para evitar data leakage. La división train/test se realiza antes de normalizar, y la normalización usa exclusivamente media y desviación típica calculadas sobre train. Esos mismos estadísticos se aplican después a test o a cada fold de validación.

Pasos realizados

16. Eliminación del columna género, ya que impone un sesgo no deseado.
17. Separación de la variable objetivo loan_status y del conjunto de características.
18. Análisis exploratorio de outliers con criterio IQR.
19. Eliminación conservadora de registros claramente sospechosos antes de la división train/test.
20. División estratificada en train y test, manteniendo la proporción de clases.
21. Codificación de variables categóricas mediante one-hot encoding con pandas.get_dummies.
22. Normalización z-score usando solo los estadísticos del conjunto de entrenamiento.
23. Conversión de los datos al formato esperado por cada modelo.
24. Balanceo opcional solo sobre train mediante undersampling o SMOTE.

El dataset no presenta valores nulos relevantes en el flujo actual, por lo que no se realiza imputación.

Tratamiento de outliers

Se añadió una función específica de EDA para mostrar un resumen de outliers por variable numérica con el criterio IQR. Después se aplicó una limpieza mínima y muy conservadora, para eliminar únicamente registros claramente sospechosos.

Regla aplicada	Motivo	Casos
person_age <= 100	Evitar edades no plausibles en el contexto del problema.	7
person_emp_exp <= 60	Evitar experiencias laborales extremadamente improbables.	10
person_income <= 1.000.000	Evitar ingresos extraordinarios que distorsionan la escala.	24

Tabla 2. Reglas conservadoras utilizadas para eliminar registros sospechosos.

La limpieza se aplica dentro de split antes de separar X e y, de forma que las filas eliminadas y sus etiquetas permanecen alineadas. En total se eliminan 31 registros, pasando de 45.000 a 44.969 ejemplos. No se eliminan todos los outliers estadísticos, ya que muchos extremos pueden ser casos financieros reales e informativos y no poseo conocimiento técnico en esa área.

Tras la limpieza, la división estratificada queda con 35.975 ejemplos en train y 8.994 en test. La proporción de clases se conserva: train contiene 27.975 ejemplos de clase 0 y 8.000 de clase 1; test contiene 6.994 ejemplos de clase 0 y 2.000 de clase 1.

Balanceo y oversampling

El proyecto compara tres formas de entrenamiento. La primera conserva el desbalanceo original. La segunda aplica submuestreo de la clase mayoritaria en train. La tercera aplica oversampling con SMOTE también solo sobre train. En validación cruzada, estas operaciones se realizan dentro de cada fold, después de separar train y validation, por lo que no se generan ejemplos sintéticos a partir de la parte de validación ni del test.

En el código, SMOTE se aplica después del one-hot encoding y la normalización, y antes de convertir los datos al formato final de cada modelo. Esto permite usarlo de forma compatible con la regresión logística, la red NumPy y la red profunda.

4. Técnicas y modelos aplicados

Modelo	Implementación	Configuración principal	Salida
Regresión logística	NumPy propio	Gradient descent, alpha=0.1, 1200 iteraciones, regularización L2 lambda=1	Vector y_pred
Red neuronal	NumPy propio	1 capa oculta de 8 neuronas, sigmoide, backpropagation, 1200 iteraciones	Vector y_pred
Red neuronal profunda	PyTorch	Arquitectura 32-16-8-salida, ReLU en capas ocultas, sigmoid final, Adam, 3000 épocas	Vector y_pred

Tabla 3. Modelos comparados en el proyecto.

La regresión logística sirve como línea base interpretable y eficiente. La red neuronal NumPy cumple el requisito de implementar una red neuronal sin depender de scikit-learn. La red profunda con PyTorch se incluye como ampliación para comprobar si una arquitectura más compleja mejora el rendimiento.

5. Metodología experimental y métricas

La evaluación se organiza en dos niveles: validación cruzada sobre el conjunto de entrenamiento y evaluación final sobre test. El test se mantiene separado hasta el final para obtener una estimación honesta del rendimiento.

Escenarios experimentales

25. Sin balanceo: se entrena con la distribución original del conjunto de entrenamiento.
26. Con undersampling: se reduce la clase mayoritaria en train para igualar el número de ejemplos de ambas clases.
27. Con oversampling SMOTE: se generan ejemplos sintéticos de la clase minoritaria únicamente a partir del conjunto de entrenamiento.

Validación cruzada

Se utiliza Stratified K-Fold con shuffle y random_state=42. La estratificación mantiene la proporción de clases en cada fold. Dentro de cada fold se vuelve a ejecutar el preprocesado, de forma que la normalización y el posible balanceo se calculan únicamente sobre la parte de entrenamiento del fold.

Métricas

Métrica	Interpretación
Accuracy	Porcentaje total de aciertos. Es útil, pero puede ser engañosa con clases desbalanceadas.
Precision	De los casos predichos como positivos, proporción que realmente era positiva.
Recall	De los positivos reales, proporción que el modelo consigue detectar.
F1	Media armónica entre precision y recall. Es la métrica principal para comparar equilibrio entre ambas.

Tabla 4. Métricas utilizadas para la evaluación.

6. Curvas de diagnóstico

Se han generado curvas para analizar la evolución del coste durante el entrenamiento y el comportamiento del rendimiento al variar el tamaño del conjunto de entrenamiento o el tamaño de la capa oculta. En el programa principal esta parte es opcional, ya que puede aumentar bastante el tiempo de ejecución.

Curvas de entrenamiento

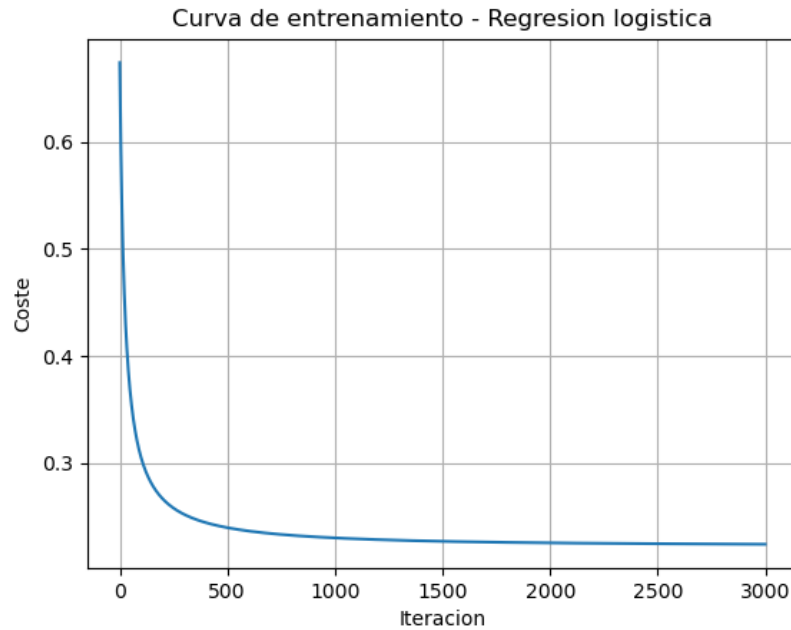


Figura 4. Curva de entrenamiento de la regresión logística.

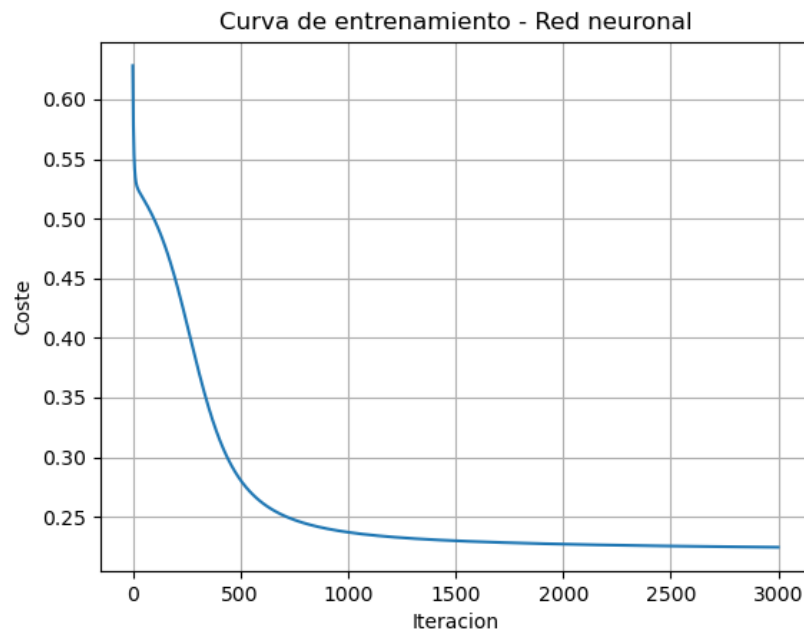


Figura 5. Curva de entrenamiento de la red neuronal NumPy.

Las curvas de coste permiten comprobar que los modelos optimizan correctamente y que el proceso de entrenamiento converge y cuándo lo hace, permitiendo ajustar Alpha para mejorar el rendimiento.

Learning curves

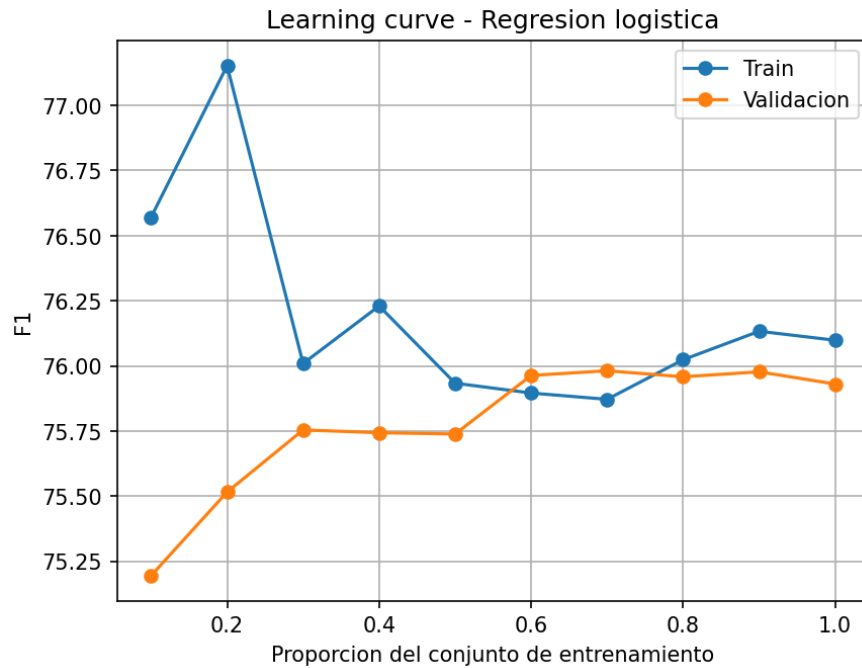


Figura 6. Learning curve de la regresión logística.

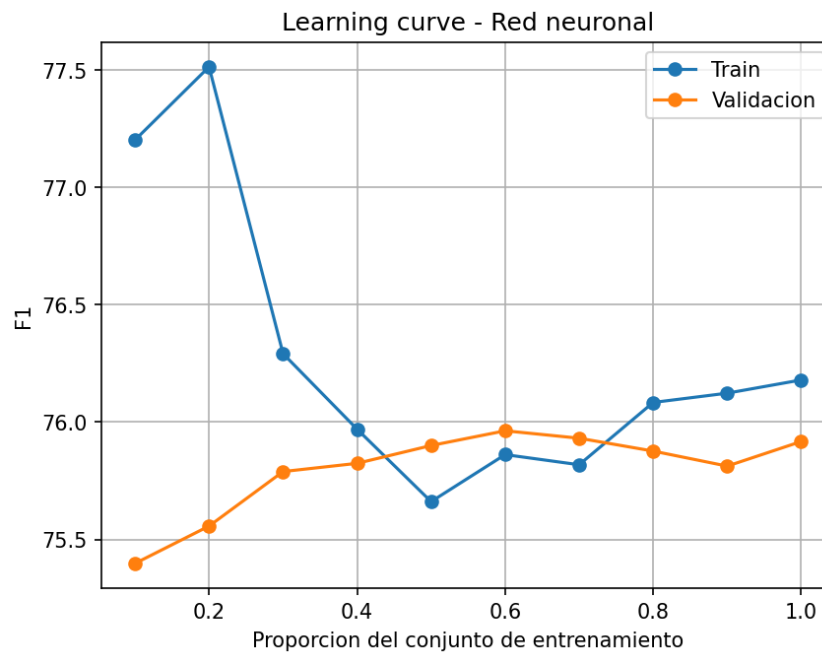


Figura 7. Learning curve de la red neuronal NumPy.

Las learning curves muestran que el rendimiento de entrenamiento y validación queda muy próximo y cambia poco al aumentar el tamaño del conjunto de entrenamiento. Este resultado es importante: el dataset aporta mucha cantidad de datos, pero los modelos alcanzan prácticamente su techo con una fracción relativamente pequeña.

Validation curve

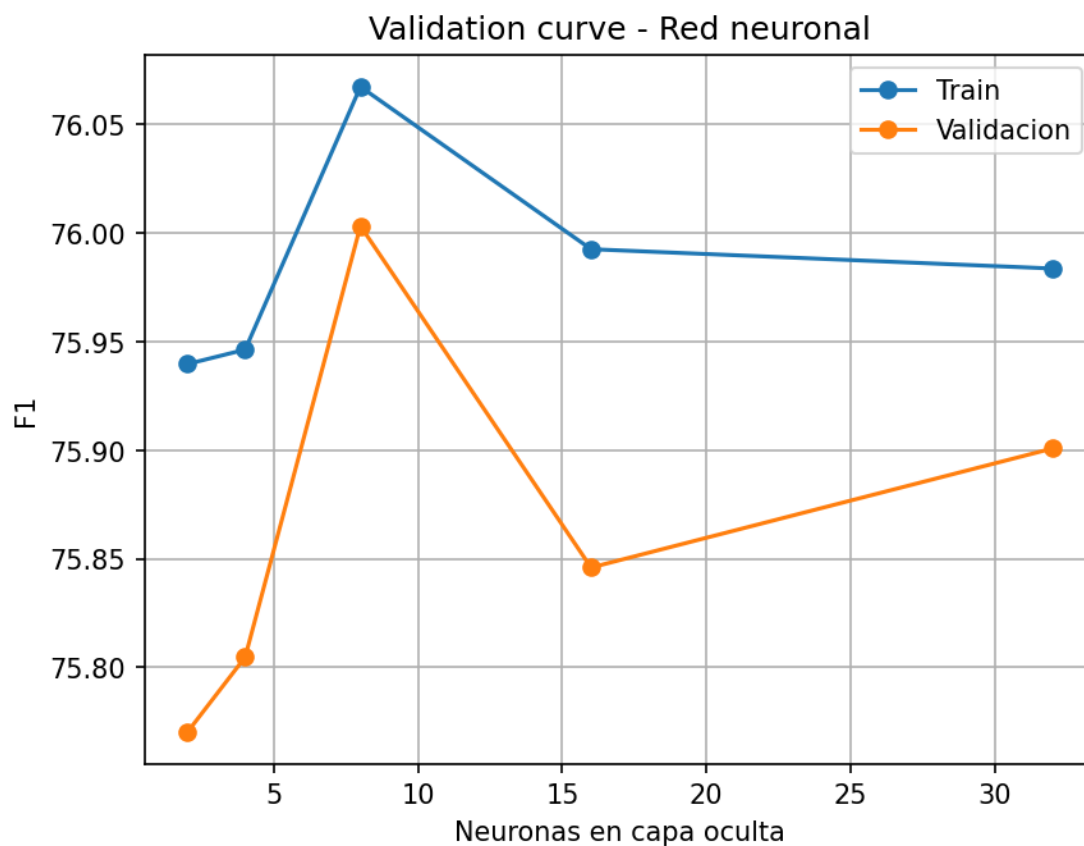


Figura 8. Validation curve variando el número de neuronas de la capa oculta.

La curva de validación muestra que aumentar el número de neuronas en la capa oculta apenas mejora el F1 de validación, las pequeñas variaciones son ruido. Por tanto, una arquitectura pequeña es más rentable y evita complejidad innecesaria.

7. Resultados experimentales

Validación cruzada sin balanceo

Modelo	Accuracy	Precision	Recall	F1
Regresión logística	89.53 ± 0.41	77.46 ± 1.07	74.62 ± 0.94	76.01 ± 0.91
Red neuronal NumPy	89.34 ± 0.46	76.05 ± 1.20	76.00 ± 0.58	76.02 ± 0.76
Red neuronal profunda	91.01 ± 0.33	83.53 ± 2.08	74.31 ± 0.54	78.61 ± 0.54

Tabla 5. Resultados medios de validación cruzada sin balanceo.

Test final sin balanceo

Modelo	Accuracy	Precision	Recall	F1
Regresión logística	89.80	78.30	74.9	76.56
Red neuronal NumPy	89.68	76.88	76.64	76.76
Red neuronal profunda	91.46	85.28	74.54	79.49

Tabla 6. Resultados finales en test sin balanceo.

Validación cruzada con undersampling

Modelo	Accuracy	Precision	Recall	F1
Regresión logística	85.36 ± 0.39	61.33 ± 0.66	92.45 ± 0.52	73.74 ± 0.60
Red neuronal NumPy	85.28 ± 0.37	61.17 ± 0.63	92.55 ± 0.56	73.66 ± 0.54
Red neuronal profunda	87.19 ± 0.33	65.72 ± 0.66	88.69 ± 0.72	75.49 ± 0.55

Tabla 7. Resultados medios de validación cruzada con submuestreo.

Test final con undersampling

Modelo	Accuracy	Precision	Recall	F1
Regresión logística	85.20	61.10	92.05	73.44
Red neuronal NumPy	85.20	61.00	92.7	73.58
Red neuronal profunda	87.56	66.34	89.5	75.20

Tabla 8. Resultados finales en test con submuestreo.

Validación cruzada con oversampling SMOTE

Modelo	Accuracy	Precision	Recall	F1
Regresión logística	85.69 ± 0.41	62.00 ± 0.74	92.11 ± 0.65	74.11 ± 0.55
Red neuronal NumPy	85.36 ± 0.37	61.36 ± 0.65	92.31 ± 0.77	73.72 ± 0.55
Red neuronal profunda	88.60 ± 0.13	69.77 ± 0.29	86.01 ± 0.62	77.04 ± 0.29

Tabla 9. Resultados medios de validación cruzada con oversampling SMOTE.

Test final con oversampling SMOTE

Modelo	Accuracy	Precision	Recall	F1
--------	----------	-----------	--------	----

Regresión logística	85.54	61.76	91.9	73.87
Red neuronal NumPy	85.21	61.04	92.55	73.56
Red neuronal profunda	89.01	70.48	87.05	77.89

Tabla 10. Resultados finales en test con oversampling SMOTE.

8. Comparativa y análisis crítico

La regresión logística y la red neuronal NumPy obtienen resultados muy similares en el escenario sin balanceo. En test, ambas se sitúan alrededor de un F1 del 76-77%. Esto indica que las relaciones del dataset son muy lineales o, al menos, suficientemente simples para que un modelo lineal regularizado capture gran parte de la estructura predictiva.

La red profunda obtiene el mejor resultado global sin balanceo, con un F1 de 79.86% en test, y también mejora en el escenario con SMOTE, donde alcanza 77.30% de F1. Aun así, la mejora no es enorme comparada con el aumento de complejidad, tiempo de entrenamiento y menor interpretabilidad.

El undersampling y SMOTE cambian el comportamiento de los modelos: aumentan mucho el recall, pero reducen la precisión y la accuracy. En este problema, el aumento de ejemplos de la clase minoritaria no mejora claramente el F1 frente al entrenamiento original. Esto encaja con lo observado en las learning curves: los modelos alcanzan su rendimiento con pocos datos y después apenas mejoran.

Idea clave
Conclusión principal: el dataset parece tener una estructura muy lineal y los modelos tocan techo pronto. La gran cantidad de datos adicionales, e incluso los datos sintéticos generados con SMOTE, no aportan una mejora sustancial porque la información útil ya está suficientemente capturada por los datos originales.

Desde un punto de vista práctico, si se busca el mejor equilibrio general, el escenario sin balanceo es el más competitivo. Si se prioriza detectar la clase positiva, undersampling o SMOTE son opciones razonables porque elevan el recall, pero a costa de generar más falsos positivos. Entre estas dos se debería elegir el undersampling, porque es menos costoso de entrenar.

9. Limitaciones y mejoras futuras

28. El tiempo de ejecución es elevado si se reproducen curvas, validación cruzada y evaluación final de tres modelos, por eso la generación de curvas se dejó como opción interactiva.
29. La red profunda aporta una mejora moderada, pero su coste computacional y menor interpretabilidad hacen que no siempre sea la opción más rentable.
30. El tratamiento de outliers se ha mantenido conservador. Podría estudiarse de forma más sistemática en una versión futura.

31. No se ha realizado clustering en el análisis exploratorio, en un futuro se podría probar con técnicas no supervisadas, para entender mejor la distribución de los datos.
32. SMOTE no mejora de forma clara el F1 general, pero podrían probarse alternativas como class weights o ajuste del umbral de decisión.
33. Se podría añadir selección de características o análisis de redundancia para simplificar el dataset.

10. Reproducibilidad

El proyecto incluye un README y un fichero requirements.txt con las dependencias necesarias. El punto de entrada principal es Proyecto/main.py. El dataset se incluye en Proyecto/data/loan_data.csv.

Elemento	Ubicación
Código principal	Proyecto/main.py
Dataset	Proyecto/data/loan_data.csv
Modelos	Proyecto/src/models/
Preprocesado	Proyecto/src/preprocess.py
Validación	Proyecto/src/validation.py
Evaluación	Proyecto/src/evaluation.py
Curvas	Proyecto/src/learning_curves.py
Dependencias	requirements.txt

Tabla 11. Elementos principales para reproducir el proyecto.

Durante la ejecución, el programa pregunta si se quiere ejecutar el EDA y si se quieren generar las curvas. Para reproducir el análisis completo se puede responder afirmativamente a ambas preguntas. Para una ejecución más rápida, se pueden omitir las curvas sin afectar a la validación cruzada ni a la evaluación final.

Se han fijado semillas aleatorias y random_state=42 en la división de datos, la validación cruzada, el submuestreo, SMOTE y los entrenamientos aleatorios. En la red profunda se usa PyTorch, por lo que pueden existir pequeñas variaciones si se ejecuta con backend MPS/GPU, aunque el comportamiento general debería mantenerse.

Para ejecutar el proyecto desde la raíz del repositorio:

```
python Proyecto/main.py
```